Open Access | https://doi.org/10.25080/ktck6444

Right Predictions, Wrong Reasons: Explanation Drift Monitoring in Production

Avik Basu [1,2]

[1]Intuit, Inc., [2]PyOpenSci

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Machine learning models in production can keep making correct predictions while silently changing the reasoning behind them. Because traditional monitoring watches input distributions and evaluation metrics, this shift often goes undetected until performance visibly degrades. We describe *explanation drift monitoring*: tracking the distribution of per-feature SHapley Additive exPlanations (SHAP) attributions over time and alerting when it moves. It is a complement to input feature drift and performance monitoring, not a replacement, and is most valuable where ground-truth labels arrive with a long delay. We describe the signals that make it measurable: per-feature Population Stability Index (PSI), a multivariate adversarial-validation score, and changes in SHAP attributions. We relate these to three failure patterns. Two case studies on the UCI Adult Income dataset delimit what the signal adds. Finally, we present `shap-monitor`, an open-source Python package implementing this workflow with low instrumentation overhead, and discuss the practical challenges of running it in production.

Keywords explainability, SHAP, model monitoring, drift detection, production machine learning

1. Introduction and Motivation

A model that keeps reporting good accuracy is easy to trust and easy to ignore. Yet a model can produce the *right predictions for the wrong reasons*. In other words, it can hold its headline metrics steady while the features driving its decisions change underneath it. When that happens, the model has quietly become a different model. A model whose behavior was never validated will eventually fail, and the eventual failure tends to be sudden and hard to diagnose.

By *wrong* we do not mean less accurate. We mean reasoning that no longer matches what was validated at deployment. A model whose attributions have moved is answering for reasons nobody reviewed, and in a regulated setting that is a problem independent of whether the headline metric has moved at all.

Most production ML monitoring is built around two families of signal, and both share a blind spot. The first is **performance monitoring**, which includes metrics such as accuracy, F1, AUC, and business KPIs. However, these metrics require the ground truth labels, which are often delayed by weeks or months. The second is **input drift monitoring**, which monitors incoming feature distributions. It needs no labels, but it answers *did the data change?* rather than *did the model's use of the data change?*.

This paper is about a third signal, **explanation drift monitoring**, which measures the change over time in *how* a model attributes its predictions to input features. We measure it by logging per-feature SHAP attributions [1] for sampled production traffic and tracking their distributions over time. When the model starts relying on different features, or relying

Published Jul 13, 2026

Correspondence to
Avik Basu
avikbasu93@gmail.com

Open Access

Copyright © 2026 Basu. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

on the same features in a different direction, the SHAP distributions move and can be detected without waiting for labels. Our contributions are:

1. A delineation of *when* explanation drift carries information that input-drift monitoring does not, and when it merely restates it. We support this with two case studies chosen to fall on opposite sides of that line: one in which input monitoring detects the shift at least as strongly as the explanation signal, and one in which input monitoring is structurally blind to it.
2. A description of concrete, low-overhead detection signals, including per-feature PSI, a multivariate adversarial-validation score, and attribution magnitude/rank/sign changes, and how to interpret them.
3. A mapping from these signals to three failure patterns that recur in production: concept drift, accuracy-preserving model-version regressions, and data-pipeline errors invisible to input monitoring.
4. An open-source library, `shap-monitor` [2], which allows users to monitor explanation drift for their models and use-cases.

2. Background

2.1. SHAP attributions

SHAP (SHapley Additive exPlanations) assigns each feature of a single prediction a contribution value derived from the Shapley value of cooperative game theory [3], distributing the gap between a model's output and a baseline expectation across the input features [1]. For a prediction $f(x)$, the attributions ϕ_i satisfy an additive decomposition,

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i, \tag{1}$$

where ϕ_0 is the base (expected) value and ϕ_i is the contribution of feature i. A positive ϕ_i pushes the prediction above the baseline; a negative one pulls it below. For tree ensembles, these attributions can be computed exactly and efficiently [4], which makes per-prediction explanation practical at production volumes.

For monitoring we are not interested in any single explanation, which is the usual per-prediction use [5]. We are interested in the *distribution* of ϕ_i for each feature across many predictions, and in how that distribution evolves over time. Two summaries are central. The **mean absolute attribution** $\mathbb{E}[|\ \phi_i\ |]$ is a global importance measure that quantifies how much feature i moves predictions, regardless of direction. The **mean signed attribution** $\mathbb{E}[\phi_i]$ captures the typical *direction* of the feature's effect. A change in either of these, in absolute terms or relative to the other features, is a change in the model's reasoning.

2.2. What existing monitoring catches, and what it misses

In practice, input drift monitoring tests each incoming feature's distribution against a reference window with a univariate statistic such as PSI or a Kolmogorov-Smirnov (KS) test, and tools such as Evidently [6] and Alibi Detect [7] make this routine. What neither family observes is the model's decision process itself. That process is a function of both the inputs and the learned model, so it can move when no single feature distribution has moved far enough to alarm and no label has yet arrived to contradict it.

2.3. Related work

Explanation distributions have been proposed as a shift-detection signal before, and we build directly on that work. Mougan et al. introduced *explanation shift*, showing on tabular data that a classifier trained to separate reference-period from current-period SHAP vectors

can be a more sensitive indicator of a performance-relevant shift than detectors operating on the inputs themselves [8], [9]. The adversarial-validation construction we describe in Section 5 is essentially their Explanation Shift Detector, and we claim no novelty for it. Attribution drift is also monitored in at least one production system: Amazon SageMaker Model Monitor compares the ranking of global SHAP importances against a training baseline using a normalized discounted cumulative gain score, and alerts when it falls below a threshold [10].

What we add on top of the related work here is threefold. First, an explicit account of *which* production failures this signal can and cannot catch, in particular separating the common case where explanation drift merely re-expresses an input shift that ordinary monitoring would already flag from the case where it is the only signal available. Second, a set of signals read *together*: per-feature PSI to localize, an adversarial score for joint shift, and magnitude, rank, and sign summaries to characterize the change, where SageMaker's monitor uses rank alone and the explanation-shift literature uses the joint classifier alone. Third, an open-source implementation that logs attributions from a live prediction path into a queryable store and is not tied to a managed platform.

3. Explanation drift

We define **explanation drift** as a statistically detectable change, over time, in the distribution of a model's SHAP attributions on production inputs. It is distinct from input feature drift in an important way. Input drift asks whether $P(X)$ has changed. Explanation drift asks whether $P(\phi(X))$ has changed, where ϕ is the explanation function induced by the *current model*. The attribution distribution couples the data and the model, so explanation drift can surface three ways: covariate shift, concept drift, or a change in the model itself. Major failure patterns develops the latter two as production failure patterns.

Crucially, none of these requires labels to detect, which makes explanation drift a *leading* signal in delayed-label regimes. Explanation drift is a complement, not a replacement. It narrows the blind spot between the other two signals rather than superseding either. Because attributions are per-feature, it also points at *which* part of the reasoning moved, which is a strong lead for root-cause analysis.

4. Detecting explanation drift

Given logged attributions for a reference period and a current period, several signals make drift measurable. They differ in whether they look at one feature at a time or at the joint distribution, and they are most useful read together.

4.1. Per-feature distribution shift (PSI)

The Population Stability Index is a long-standing measure of how much a distribution has moved between two samples, widely used in credit risk to monitor scorecards [11]. It bins a reference distribution, compares the proportion of mass each bin receives under the current distribution, and sums the divergence:

$$\mathrm{PSI} = \sum_{b=1}^{B} (c_b - r_b) \ln \frac{c_b}{r_b}, \tag{2}$$

where r_b and c_b are the reference and current proportions in bin b. Applied to the SHAP values of a feature, PSI quantifies how much that feature's *contribution* distribution has shifted. The conventional banding (below 0.1 is stable, 0.1–0.25 warrants investigation, above 0.25 indicates significant shift) was developed for raw scorecard distributions, and we carry it over to attribution distributions unchanged; it is a useful default rather than a

calibrated result, and Section 12 records an important caveat on reading large values. PSI is cheap, interpretable, and per-feature, which makes it a good first-pass alarm and a good way to localize drift to specific features.

4.2. Multivariate shift (adversarial validation)

Per-feature tests miss joint shifts, i.e., combinations of attributions can move even when no single feature's marginal PSI looks alarming. *Adversarial validation* captures this. We label reference-period attribution vectors as class 0 and current-period vectors as class 1, train a simple classifier to tell them apart, and measure its cross-validated AUC. Adversarial validation is a general technique [12]; applying it to attribution vectors as a drift detector is due to Mougan et al., who introduce it as the *Explanation Shift Detector* [8], [9]. An AUC near 0.5 means the two periods are statistically indistinguishable; an AUC approaching 1.0 means the model's reasoning in the two periods is easily separable, i.e., strong evidence of drift. The classifier's feature importances additionally rank *which* attribution dimensions drive the separation, giving a multivariate complement to per-feature PSI.

4.3. Magnitude, rank, and sign changes

Beyond distributional distance, three interpretable summaries describe the *nature* of a shift. The **change in mean absolute attribution** says whether a feature became more or less influential overall. The **change in importance rank** says whether the model's priority ordering of features was reshuffled. And a **sign flip**, i.e., a change in the typical direction of a feature's contribution, means a feature that used to push predictions one way now pushes them the other. Sign flips on important features are a high-signal indication that relationships learned during training no longer hold.

5. Major failure patterns

Explanation drift is worth monitoring because it catches failure modes that the other signals are structurally poor at catching. Three recur often enough to be worth naming.

5.1. Concept drift: same features, shifted relationships

In concept drift the relationship between features and target changes while the inputs may look similar [13]. A feature that was once weakly predictive becomes strongly predictive, or reverses direction. Input monitoring can be quiet, because the marginal feature distributions need not move much. Performance monitoring eventually catches it, once labels arrive. Explanation drift sits in between: as the model encounters inputs whose learned relationships no longer match reality, its attribution magnitudes and signs shift, and PSI on the affected SHAP distributions rises before the labels confirm the damage.

5.2. Model-version regression: same accuracy, different reasoning

Model retraining and redeployment is routine, and validation usually gates on aggregate metrics. A new version can match the old one's accuracy on a holdout set while arriving at its predictions through materially different reasoning, e.g., leaning on a leak-prone proxy feature, say, or down-weighting a feature the business considers essential. Because the headline metric is preserved, this passes standard checks silently. Comparing the SHAP attribution distributions of the two versions on the *same* inputs surfaces it directly: a high adversarial-validation AUC between versions, or large per-feature rank and sign changes, reveals that the reasoning regressed even though the accuracy did not.

5.3. *Data-pipeline errors invisible to input monitoring*

Some of the most damaging production incidents are upstream data bugs: a feature silently defaulting to a constant, a units change, a join that drops rows, a transformation applied in the wrong order. When such a bug pins a feature to a plausible value, its marginal distribution can stay within range and input monitoring stays quiet, but the model's *use* of that feature collapses or distorts, which the attribution distribution registers immediately. Because explanation drift is per-feature, it both detects the problem and points at the feature whose pipeline to inspect.

6. Case studies

We report two studies, chosen to sit on opposite sides of the question the previous section raises: *when does watching attributions tell you something that watching inputs would not?* Study A is a covariate shift, the scenario in which explanation drift is most often demonstrated. It is also, as we show, one in which input monitoring detects the same event at least as decisively. Study B removes input drift entirely by construction, leaving explanation drift as the only label-free signal available.

Both use the UCI Adult Income dataset [14], a standard tabular benchmark predicting whether annual income exceeds \$50K, and a LightGBM classifier [15] with exact attributions from `TreeExplainer` [4]. Every number, table, and figure below is regenerated by the two notebooks in `examples/paper_notebooks/` of the `shap-monitor` repository [2], one per study. Both fetch the dataset directly, so no manual data preparation is required.

Because attributions carry sampling variance, every adversarial-validation figure we report is the mean over 20 independent subsamples of 4,000 rows per window, with intervals giving the 2.5th and 97.5th percentiles across those repetitions; PSI intervals come from 200 bootstrap resamples. We deliberately subsample *without* replacement. A with-replacement bootstrap duplicates rows, and because the adversarial AUC is cross-validated, a duplicated row can appear in both a training and a validation fold; the classifier then memorizes it. That artifact inflated a genuine no-drift control from 0.50 to roughly 0.61 in our first attempt, and anyone reproducing this measurement should be aware of it.

6.1. *Study A: a stable score over a shifting population*

We train on the younger population (age ≤ 45) and treat the older population (age > 45) as drifted production traffic, mirroring a model developed on one population and deployed to serve another. The reference window is the held-out validation split ($n = 10,290$); the current window is the older population ($n = 14,544$).

Performance barely moves. F1 is 0.710 on the reference data and 0.721 on the drifted data. Watching F1 alone, and recalling that in production those labels would not exist for weeks, nothing would prompt an investigation.

The attribution signal fires, and so does input monitoring. The adversarial-validation AUC over SHAP attributions rises from 0.499 [0.489, 0.512] on a no-drift control to 0.967 [0.965, 0.971] between the two periods. The same statistic computed on the *raw inputs*, however, reaches 1.000 [1.000, 1.000]. On this shift, input-drift monitoring is not merely also triggered: at the level of the joint distribution it separates the two periods perfectly, and more decisively than the explanation signal does. Figure 1 shows this in its left panel.

A deliberate demographic shift is exactly the regime input monitoring was designed for, and explanation drift adds no *global* detection power here. Taken alone, Study A establishes the weaker of the available claims: that a model's attribution behavior can change substantially while its accuracy does not.

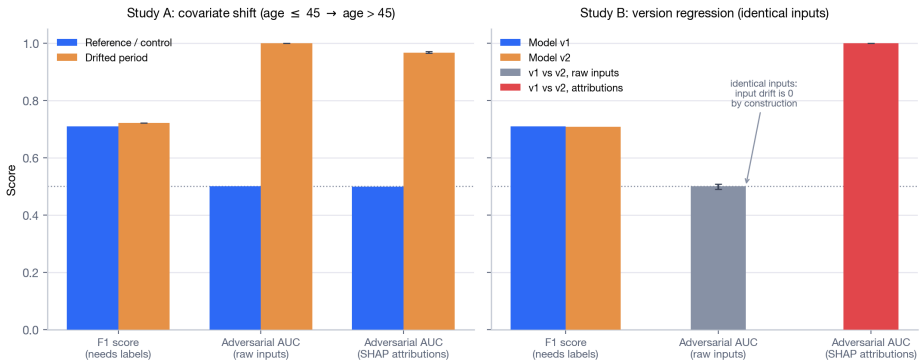


Figure 1. *Two regimes for the same signal.* ***Left (Study A):*** *under a demographic covariate shift the model's F1 is flat (0.710 arrow.r 0.721), so performance monitoring sees nothing, but* both *the raw-input and the SHAP adversarial scores detect the shift, the raw inputs more strongly (1.000 vs. 0.967). Explanation drift adds no global detection power in this regime.* ***Right (Study B):*** *two model versions scored on identical inputs. Raw-input drift is zero by construction and the raw adversarial score sits at chance (0.500), while the attribution distributions are perfectly separable (1.000) at effectively unchanged accuracy. Error bars are 2.5–97.5 percentile intervals over 20 repeated subsamples.*

The per-feature view is where the two signals disagree. Figure 2 plots each feature's PSI in attribution space beside its PSI in input space, and the aggregate picture above conceals a systematic divergence.

For four features the input distribution is quiet while the attributions are not. The clearest is `race`: its raw PSI is 0.008, far below the 0.1 "warn" band, while its attribution PSI is 0.752, far above the 0.25 "alert" band, a ratio of roughly 91. The racial composition of the traffic barely moved; the model's *use* of race changed substantially. `native-country` shows the same pattern (0.426 vs. 0.038), as do `hours-per-week` (0.118 vs. 0.008) and `capital-loss` (0.176 vs. 0.000). An input-drift dashboard would show nothing actionable on any of these four; in attribution space all four clear the warn threshold and `race` and `native-country` clear the alert threshold.

This is the sense in which the predictions here are right for the wrong reasons. The model's F1 *improved* slightly across this shift, so nothing in the performance signal argues for intervention, and the racial composition of the traffic barely moved, so nothing in the input signal does either; yet the model's reliance on `race` shifted by two orders of magnitude more than that composition did. Whether the change is harmful is a question for review; that it would have gone unreviewed is the failure.

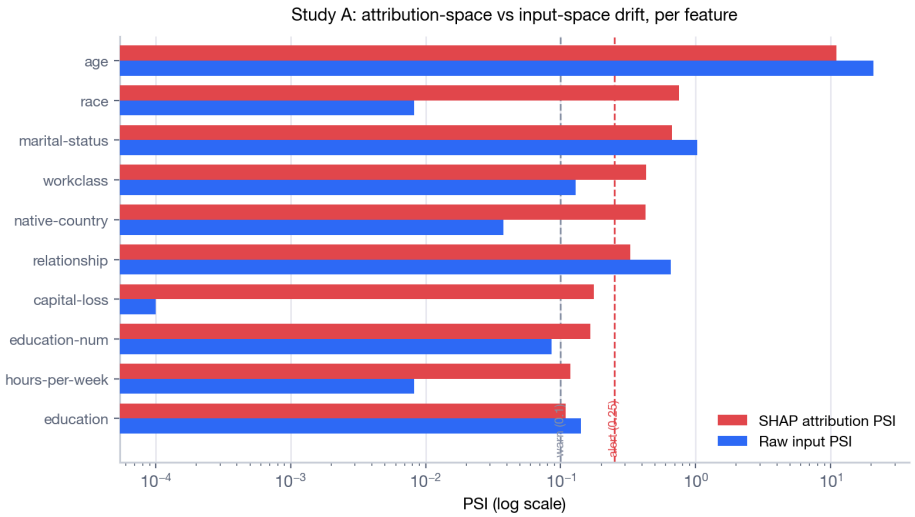


Figure 2. *Per-feature PSI in attribution space (red) against input space (blue), reference vs. drifted period, log scale. Dashed guides mark the conventional 0.1 (warn) and 0.25 (alert) thresholds. The two views agree on the features the shift acted on directly (`age`, `marital-status`, `relationship`, all larger in input space), and disagree sharply on `race`, `native-country`, `hours-per-week`, and `capital-loss`, where the inputs are stable but the model's use of them is not. Four features falling below the warn band in both views (`capital-gain`, `fnlwgt`, `occupation`, `sex`) are omitted, since neither kind of monitoring would act on them.*

`shap-monitor` reports a +33% change in `age` mean absolute attribution, +82% for `workclass`, +51% for `capital-gain`, and sign flips on six features including `marital-status` and `workclass`, meaning their typical contribution direction reversed.

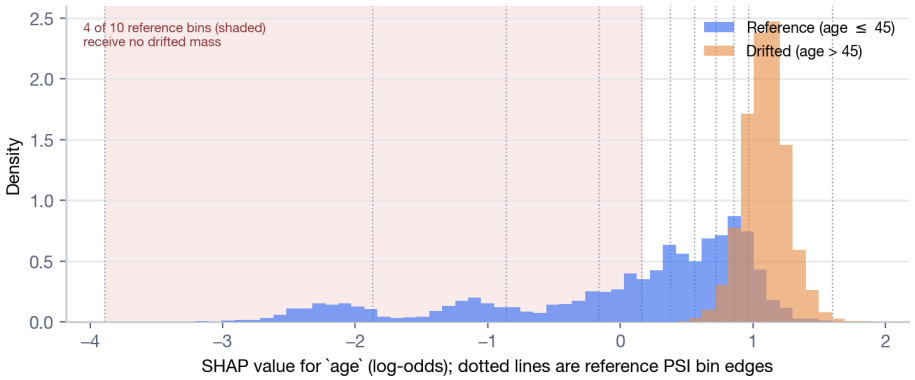


Figure 3. *Distribution of `age` SHAP values, reference (blue) vs. drifted (orange). The reference distribution is broad and bimodal; under the shift it collapses to a tight, strongly positive cluster, so `age` becomes a consistent large positive driver for the older population. Dotted lines are the ten reference quantile bin edges PSI is computed over, and the shaded span marks the four bins that receive no drifted mass at all, the mechanism behind the inflated PSI discussed in Section 12.*

6.2. Study B: same inputs, same accuracy, different reasoning

We reuse Study A's training split, the same 24,008 younger-population training rows, and fit several model versions on it, then score every version on the same 10,290 validation rows. The older population plays no part here. Because all versions consume byte-identical inputs, per-feature input PSI is exactly 0 for all fourteen features under *every* construction, and the adversarial score on the raw inputs sits at chance, 0.500 [0.490, 0.509]. This is not a small number that might have been larger under a different shift; there is no input difference

to detect. For calibration, the same no-shift measurement in attribution space gives 0.499 [0.489, 0.512].

Reporting a single constructed regression would invite the objection that it is the one setting where the signal happens to work. We therefore report a family of four version pairs, summarized in Table 1, spanning a true null control, an un-engineered change of training configuration, and two deliberate feature deprioritizations implemented with LightGBM's per-feature split-gain penalty.

Table 1. *Four version-2 constructions, all scored on the same 10,290 rows as version 1 (F1 = 0.710). Raw-input PSI is exactly zero for every feature in every row, so input monitoring cannot fire. "Flagged" counts features whose attribution PSI reaches the 0.25 alert band.*

Name	v2 construction	F1	ΔF1	Agreement	SHAP AUC	Flagged
Control	identical retrain	0.710	+0.000	100%	n/a	**none**
Retune	new hyperparameters	0.695	-0.014	98.2%	1.00	10 of 14
Demote-rel	`relationship` denied split gain	0.709	-0.001	98.7%	1.00	3
Demote-edu	`education-num` denied split gain	0.705	-0.004	98.7%	1.00	2

Control stays silent, and accuracy monitoring passes everything else. Refitting with identical data and settings reproduces version 1 exactly. Across the three genuine regressions, meanwhile, the largest F1 change is 1.4 points and the smallest is under a *tenth* of a point, so a gate on aggregate metrics admits all of them.

Demote-rel relocates reasoning without disturbing accuracy. Here F1 moves by -0.001 while the attribution structure is rearranged: `relationship` collapses from a mean absolute attribution of 0.579 to 0.000 and falls from rank 3 to rank 14, its correlated substitute `marital-status` absorbs the role, rising from 0.795 to 1.414 (+78%) and from rank 2 to rank 1, and `sex` declines from 0.096 to 0.057. Attribution PSIs are 20.72, 3.14, and 0.99 respectively; every other feature stays below the alert band. The alerting policy would page on exactly the three features involved and stay quiet on the remaining eleven. Figure 4 shows the full picture.

Retune shows that localization degrades when the change is diffuse. It is the least contrived of the three: no penalty is applied, only a different training configuration, which is the most common thing to differ between two production versions. It is detected, but it flags 10 of 14 features, because a broad configuration change moves reasoning broadly rather than relocating one feature's role. It is also the row where F1 drops most (-0.014), so it is the weakest of the three on the accuracy-preservation axis. We report it because its realism is the point, and its diffuseness is a useful warning: the per-feature signals localize a targeted regression well and a systemic one poorly.

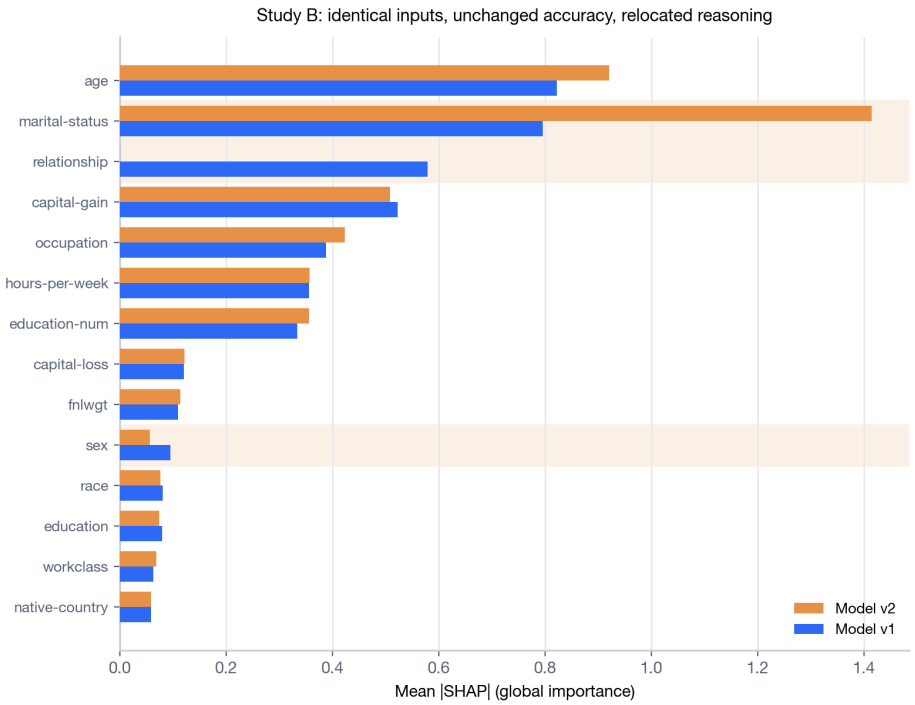


Figure 4. *Study B, `demote-relationship` construction: mean absolute SHAP attribution per feature for version 1 (blue) and version 2 (orange), computed on identical inputs. Shaded rows mark the affected group. `relationship` collapses to zero while its correlated substitute `marital-status` absorbs the role. Overall accuracy and every input distribution are unchanged; the reasoning has been relocated.*

This is the model-version regression of Section 8, demonstrated rather than described, and it is the case in which explanation drift is not a complement to the existing signals but the only label-free signal available at all. The same argument applies to the pipeline-error pattern of Section 9, where a feature pinned to a plausible constant leaves the marginal distribution in range while the model's use of it collapses.

6.3. A caveat on reading large PSI values

The `age` attribution PSI of 11.07 [10.90, 12.49] in Study A deserves comment, because a value two orders of magnitude above the alert threshold invites being read as a magnitude when it should be read as a saturation flag.

`shap-monitor` bins the reference distribution into ten quantile bins, so each reference bin holds about 10% of the mass by construction and the r_b term cannot approach zero. Under the null this behaves well: the largest attribution PSI in our no-drift control is 0.0079. The instability is on the other side of the ratio. When the current distribution collapses into a narrow region, as `age` does in Figure 3, reference bins receive *no* current mass, and c_b must be floored at some ε to keep the logarithm finite. Four of the ten bins are empty here, and they contribute 75% of the total. Since each empty bin contributes roughly $-r_b \ln(\varepsilon / r_b)$, the total is a direct function of that constant: holding the data fixed and varying ε from 10^{-10} to 10^{-3} moves the reported PSI from 11.07 to 4.61.

PSI therefore remains sound for *detection* and for *ranking* features by how far their attributions moved, which is what a monitoring system needs. But a value far above the alert band should be reported as saturated rather than quoted as a quantity, and such values are not comparable across implementations that choose different flooring constants. Where a stable magnitude is wanted, a bounded divergence such as Jensen–Shannon avoids the

artifact entirely; we report PSI here for continuity with the banding practitioners already know.

7. Open-source implementation

`shap-monitor` [2] is an open-source (Apache-2.0) Python package that implements this workflow with a deliberately small surface area: a logger to capture attributions in production, a storage backend, and an analyzer to compute drift. It builds on SHAP [1], scikit-learn [16], [17], NumPy [18], and pandas [19]. The repository's `examples/lightgbm_example.ipynb` introduces the API step by step.

7.1. Logging attributions in production

The `SHAPMonitor` wraps any SHAP explainer and logs attributions for a configurable fraction of traffic. Sampling keeps the overhead bounded: explanation cost scales with the number of explained rows, so a sample rate well below 1.0 is appropriate for high-volume services.

```
from shapmonitor import SHAPMonitor
import shap

monitor = SHAPMonitor(
    explainer=shap.TreeExplainer(model),
    data_dir="/var/log/shap",
    sample_rate=0.05,          # explain 5% of traffic
    model_version="prod-v2.1",
)

# In the prediction path:
predictions = model.predict(X)
monitor.log_batch(X, predictions)   # samples, explains, and persists
```

`log_batch` samples the batch, computes SHAP values for the sampled rows, and persists them. The logging call is best wrapped so that a monitoring failure can never take down the prediction path.

7.2. Storage

Attributions are written as Parquet [20], partitioned by date in a Hive-style layout (`date=YYYY-MM-DD/`) so that time-range queries prune to the relevant partitions, with optional partitioning by model version. Each record carries a timestamp, a batch UUID, the model version, the per-feature SHAP values, and optionally the feature values and predictions. The columnar format keeps storage compact and is directly readable by pandas, DuckDB, and Spark. The storage layer is a small protocol, so a different backend (an object store or a warehouse) can be substituted without touching the logging or analysis code.

7.3. Analyzing drift

The `SHAPAnalyzer` reads logged attributions and computes the signals described above. Comparisons can be made between two time periods, two specific batches, or two model versions:

```python
from datetime import datetime
from shapmonitor.analysis import SHAPAnalyzer
from shapmonitor.backends import ParquetBackend

analyzer = SHAPAnalyzer(ParquetBackend("/var/log/shap"))

# Per-feature PSI, magnitude/rank/sign changes between two windows.
drift = analyzer.compare_time_periods(
    (datetime(2026, 5, 1), datetime(2026, 5, 8)),    # reference
    (datetime(2026, 5, 8), datetime(2026, 5, 15)),   # current
)
alerts = drift[drift["psi"] >= 0.25]          # significant shifts
flips = drift[drift["sign_flip"]]             # reversed contribution direction

# Multivariate adversarial-validation score for the same windows.
adv = analyzer.compare_adversarial(
    (datetime(2026, 5, 1), datetime(2026, 5, 8)),
    (datetime(2026, 5, 8), datetime(2026, 5, 15)),
)
print(adv.attrs["adversarial_auc"])           # 0.5 = no drift, 1.0 = max drift
```

The comparison returns, per feature, its PSI, its mean absolute attribution in each period, the percentage change in importance, the change in importance rank, and whether its contribution direction flipped, exactly the quantities reported in the case studies. To compare two deployed models scored on the same traffic, which is the version-regression pattern of Section 8, `compare_versions` performs the same analysis keyed on the logged model version rather than on time.

7.4. Command line

For operational use the same analysis is available from a command-line interface, which is convenient for cron-driven reports and for piping machine-readable output into an alerting system:

```bash
# Human-readable drift report between two windows.
shapmonitor report drift --data-dir /var/log/shap \
    --ref last-14d..last-7d --curr last-7d..now

# Machine-readable output for an alerting rule.
shapmonitor report drift --data-dir /var/log/shap --json \
    | jq '.features[] | select(.psi >= 0.25)'
```

A simple and effective alerting policy is threshold-based: page when any important feature's SHAP PSI crosses 0.25, or when the adversarial-validation AUC between the current window and a fixed baseline exceeds a chosen level (for example 0.8). Restricting per-feature alerts to features above a minimum mean absolute attribution avoids noisy alarms on features the model barely uses.

8. Challenges and Limitations

Explanation drift monitoring is a useful signal, but it is not free and not a panacea.

Explainer cost. Computing the explanations dominates the cost, not storing or analyzing them. Exact attributions are cheap for tree models, where `TreeExplainer` is fast enough to run inline on a sampled fraction of requests, but expensive for model-agnostic explainers such as `KernelExplainer`. Sampling and out-of-band computation (logging the inputs and explaining them in a periodic batch job, so the prediction path is never blocked) mitigate this, but the cost must be budgeted, and very low sample rates trade statistical power for cheapness.

Baseline choice. Drift is always measured *relative to a reference*. A poorly chosen baseline (too short, unrepresentative, or itself already drifted) produces misleading comparisons. Seasonal traffic in particular calls for a seasonally aware reference rather than a naive trailing window.

Threshold tuning. The PSI bands are conventions, not laws, and the right alerting thresholds depend on the model, the feature, and the tolerance for false alarms. Thresholds should be calibrated against a quiet period and revisited as the system evolves.

Attribution stability and correlation. SHAP attributions can be sensitive to correlated features and to explainer configuration, so a portion of measured drift may reflect attribution variance rather than genuine reasoning change. Adequate sample sizes per window and a fixed explainer configuration reduce, but do not eliminate, this. Every headline figure we report therefore carries an interval over repeated subsamples rather than being quoted as a point estimate.

Reading the adversarial score. Because $\phi(X)$ depends on X, an attribution-space statistic moves whenever the inputs move, so a high adversarial AUC is not on its own evidence of a reasoning change: in Study A the raw inputs are separable at AUC 1.000, and most of the attribution-space separability is inherited rather than added. Reporting the input-space statistic alongside it, as we do throughout, is what disentangles them, and the case for explanation drift as an independent signal rests on Section 11, where the inputs are identical by construction. The score also saturates (a systematic change across ten thousand rows reaches 1.000 whether the underlying regression is total or partial), making it a sensitive detector but a poor measure of severity. The per-feature magnitude, rank, and sign changes should carry the diagnostic weight.

Scope of the evidence. Both case studies use a single public benchmark, and the version regressions in Section 11 are deliberately constructed. Together they establish that this class of failure exists and that the other two monitoring signals are structurally unable to see it; they do not establish how frequently it arises in production, which a public dataset cannot show.

It is a leading indicator, not truth itself. Explanation drift tells you the model's reasoning changed; it does not, by itself, tell you whether that change is harmful. It is a high-quality lead that warrants investigation, ideally triangulated against input drift and, once available, performance, not an automatic trigger for rollback.

9. Conclusion and future work

Production models can keep their accuracy while quietly changing the reasoning behind their predictions, and the standard monitoring stack, input drift plus delayed-label performance, has a structural blind spot for exactly this. Tracking SHAP attribution distributions over time closes part of that gap with a label-free signal that, because it is per-feature, both detects drift and localizes it.

Our two studies delimit where it helps. Under a demographic covariate shift (Section 10), input-drift monitoring separated the two periods more decisively than the attribution view did; what the attribution view added was per-feature, on `race` and three others where the inputs stayed quiet while the attributions crossed the warn band. In Section 11, versions scored on byte-identical inputs preserve accuracy while relocating a rank-3 feature's contribution onto a correlated substitute, so input and performance monitoring are both silent and explanation drift is the only signal that fires. It stays quiet on a true null retrain.

The `shap-monitor` package provides an open-source implementation. Promising directions for future work include asynchronous logging, additional storage backends, and alerting

that fuses explanation drift with input-drift and performance signals into a single, better-calibrated alarm. We hope explanation drift monitoring becomes a standard third pillar of production ML observability, alongside the two it complements.

References

[1] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774. doi: 10.48550/arXiv.1705.07874.

[2] A. Basu, "shap-monitor: Production monitoring of SHAP explanations over time." [Online]. Available: https://github.com/ab93/shap-monitor

[3] L. S. Shapley, *A Value for n-Person Games*. Princeton, NJ: Princeton University Press, 1953, pp. 307–317. doi: 10.1515/9781400881970-018.

[4] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[5] A. Basu, "Explaining ML predictions with SHAP: Intuition, use cases, and best practices," in *Proceedings of the 24th Python in Science Conference*, 2025, pp. 14–37. doi: 10.25080/mhum9729.

[6] Evidently AI Team, "Evidently: An open-source framework to evaluate, test and monitor ML and LLM-powered systems." [Online]. Available: https://github.com/evidentlyai/evidently

[7] J. Klaise, A. Van Looveren, C. Cox, G. Vacanti, and A. Coca, "Monitoring and explainability of models in production," in *ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, 2020. doi: 10.48550/arXiv.2007.06299.

[8] C. Mougan, K. Broelemann, G. Kasneci, T. Tiropanis, and S. Staab, "Explanation Shift: Detecting distribution shifts on tabular data via the explanation space," in *NeurIPS 2022 Workshop on Distribution Shifts: Connecting Methods and Applications*, 2022. doi: 10.48550/arXiv.2210.12369.

[9] C. Mougan, K. Broelemann, D. Masip, G. Kasneci, T. Thiropanis, and S. Staab, "Explanation Shift: How Did the Distribution Shift Impact the Model?." [Online]. Available: https://arxiv.org/abs/2303.08081

[10] D. Nigenda *et al.*, "Amazon SageMaker Model Monitor: A System for Real-Time Insights into Deployed Machine Learning Models," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 3671–3681. doi: 10.1145/3534678.3539145.

[11] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ: John Wiley & Sons, 2006.

[12] J. Pan, "Adversarial Validation, Part One." [Online]. Available: http://fastml.com/adversarial-validation-part-one/

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014, doi: 10.1145/2523813.

[14] B. Becker and R. Kohavi, "Adult." UCI Machine Learning Repository, 1996. doi: 10.24432/C5XW20.

[15] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, doi: 10.48550/arXiv.1201.0490.

[17] L. Buitinck *et al.*, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122. doi: 10.48550/arXiv.1309.0238.

[18] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[19] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[20] Apache Software Foundation, "Apache Arrow and the Parquet columnar storage format." [Online]. Available: https://parquet.apache.org/